

Historical VAR Model — Technical Specification

Document type: Model Development Technical Specification Version: 1.0 Date: 2026-02-28 Status: Implementation Complete Scope: Value-at-Risk and Expected Shortfall for a European Equities Options Portfolio

Table of Contents

1. Overview
 2. System Architecture
 3. Data Layer
 4. Options Pricing — Black-Scholes Model
 5. Volatility Surface Construction
 6. Scenario Construction
 7. Portfolio Revaluation
 8. Risk Measures — VAR and ES
 9. Method A — Plain Historical Simulation
 10. Method B — Age-Weighted Historical Simulation
 11. Method D — Factor-Based VAR
 12. Stressed VAR — Basel 2.5
 13. Rolling Backtest
 14. Statistical Tests
 15. Trade Lifecycle Management
 16. Implementation Engineering Notes
 17. Model Limitations and Known Constraints
 18. Parameter Reference
 19. Module Reference
-

1. Overview

This document provides a full technical specification of the Historical Simulation Value-at-Risk (HS-VAR) model implemented for a portfolio of European equity options.

The model computes Value-at-Risk (VAR) and Expected Shortfall (ES / CVaR) using four methodologies:

Label	Method	Regulatory alignment
A	Plain Historical Simulation (equal-weight)	Basel II/III internal model
B	Age-Weighted Historical Simulation (λ -decay)	Basel II/III extension
D	Factor-Based VAR (systematic + idiosyncratic)	Multi-factor extension

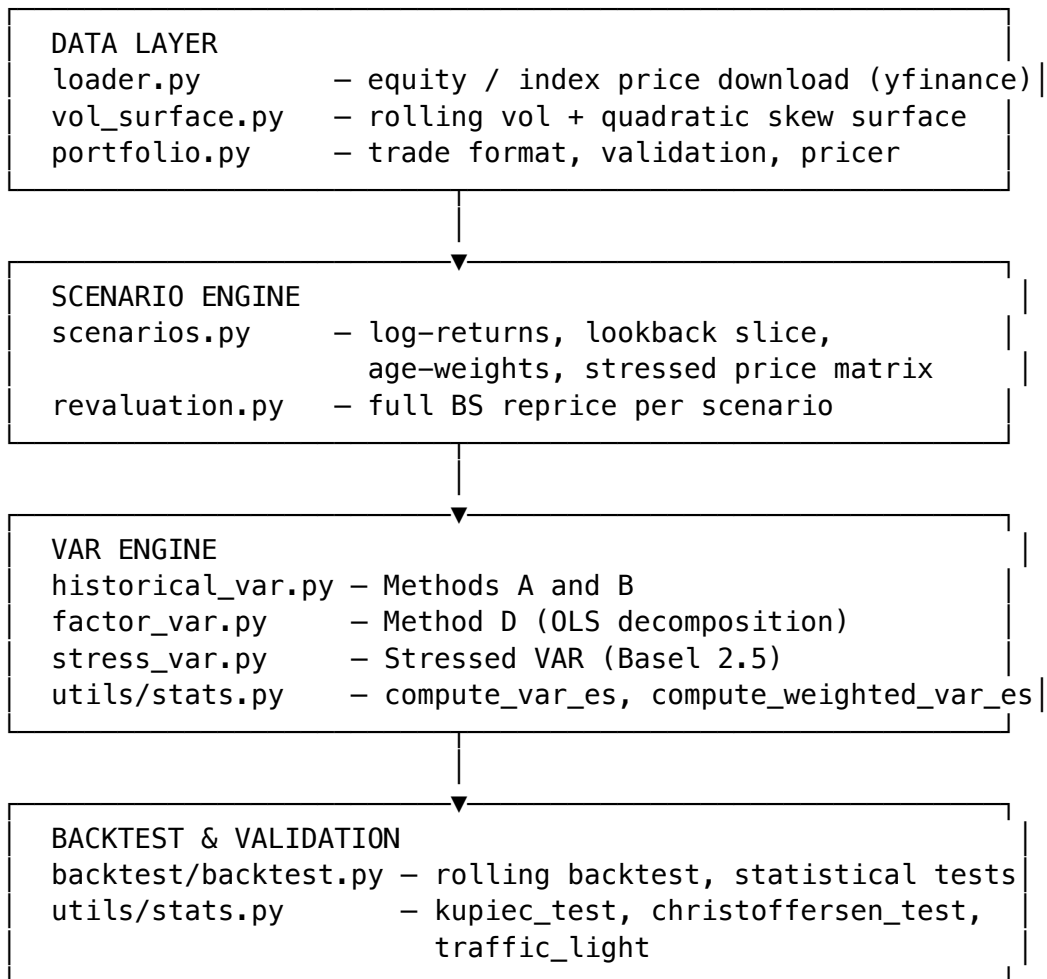
Label	Method	Regulatory alignment
SVaR	Stressed VAR	Basel 2.5 / FRTB IMA

All methods share a common pricing kernel (Black-Scholes with a synthetic vol surface) and produce both VAR and ES at configurable confidence levels (default: 95% and 99%) for 1-day and 10-day horizons.

Regulatory context

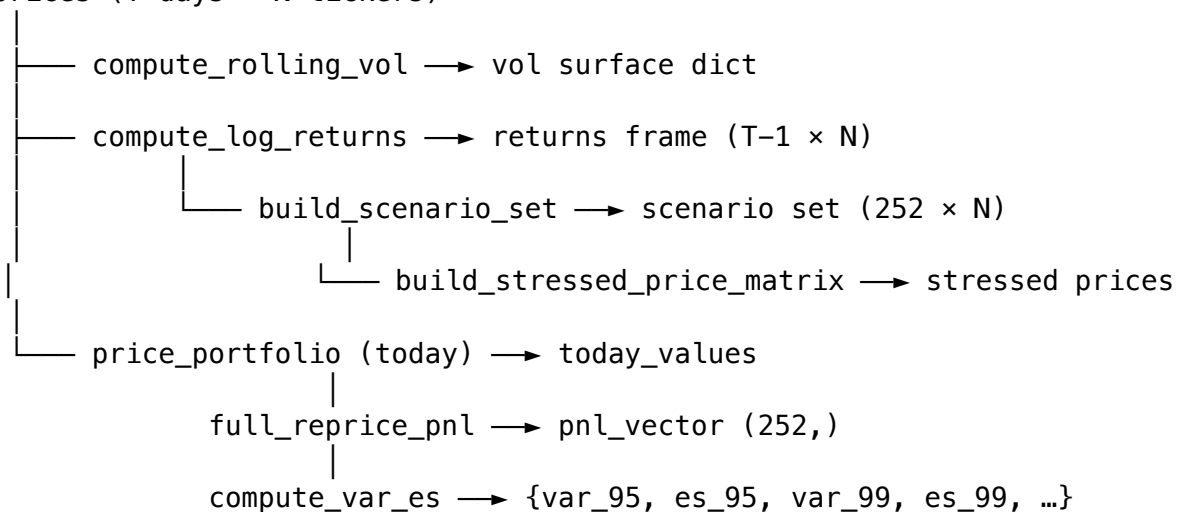
- Basel II/III Internal Model Approach (IMA): 1-day 99% VAR, 250-day backtest, Kupiec Proportion-of-Failures test, Christoffersen independence test, Basel traffic-light classification.
- Basel 2.5 / FRTB: Supplemental Stressed VAR over the worst 252-day rolling window.
- FRTB IMA (informational): ES at 97.5%, though the implementation supports any confidence level.

2. System Architecture



Data flow summary

Market prices (T days $\times$ N tickers)



3. Data Layer

3.1 Equity and index prices

Historical adjusted close prices are downloaded from `yfinance`. For the production configuration, the intended universe is 25 single-name equities (technology, financials, healthcare, consumer, energy/industrials) plus the equity indices (^GSPC, ^NDX, ^RUT).

The demo notebook uses synthetic GBM prices to ensure the model can run offline and reproducibly.

Price data specification:

Property	Value
Field	Adjusted close (splits and dividends accounted for)
Frequency	Daily business days
Required history	5 years (~1 300 business days)
Missing data	Forward-fill then back-fill
Warning threshold	> 5% missing data per ticker

Download resilience: `loader.py` implements exponential-backoff retry logic (3 attempts, delays of 1 s / 2 s / 4 s) for `ConnectionError`, `TimeoutError`, and `OSError`. `ValueError` (empty response) is raised immediately without retry.

3.2 Portfolio format

Trades are stored in a pandas DataFrame (or CSV file) with the following schema:

Column	Type	Constraint
trade_id	str	Unique

Column	Type	Constraint
underlying	str	Must match a ticker in prices
option_type	str	"call" or "put"
strike	float	> 0
expiry	date	Strictly after trade_date
notional	float	> 0 (contracts × multiplier)
position_sign	int	+1 (long) or -1 (short)
premium	float	Fair-value BS price at trade date
trade_date	date	

Portfolio position value for a single trade:

$\text{position_value} = \text{unit_price} \times \text{notional} \times \text{position_sign}$

where unit_price is the Black-Scholes price per unit (see Section 4).

3.3 Synthetic portfolio generator

`generate_portfolio` creates a reproducible synthetic portfolio for testing and demonstration. It draws uniformly from: - underlyings: all tickers in `spot_prices` - option types: call / put (equal probability) - position signs: long / short (equal probability) - moneyness: $\text{strike} = \text{spot} \times U[0.80, 1.20]$ - expiry: 63, 126, 189, or 252 business days (uniform choice) - notional: 100, 200, or 500 (uniform choice)

Premiums are computed at fair value using the vol surface on `as_of`.

4. Options Pricing — Black-Scholes Model

4.1 Pricing formula

All options are European. The Black-Scholes closed-form price is used throughout.

Call:

$$C = S N(d_1) - K e^{-rT} N(d_2)$$

Put:

$$P = K e^{-rT} N(-d_2) - S N(-d_1)$$

where:

$$d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}$$

Notation:

Symbol	Description
S	Current spot price
K	Strike price
T	Time to expiry in years (calendar days / 365.25)
r	Continuously compounded risk-free rate
σ	Implied volatility (annualised)
$N(\cdot)$	Standard normal CDF

Expiry handling: When $T \leq 0$ the option is valued at intrinsic: $\max(S - K, 0)$ for calls, $\max(K - S, 0)$ for puts.

4.2 Greeks

All four first-order Greeks are computed analytically:

Greek	Formula
Delta ($\partial V / \partial S$)	Call: $N(d_1)$ · Put: $N(d_1) - 1$
Gamma ($\partial^2 V / \partial S^2$)	$N'(d_1) / (S \sigma \sqrt{T})$ — same for calls and puts
Vega ($\partial V / \partial \sigma$)	$S N'(d_1) \sqrt{T}$ — same for calls and puts
Theta ($\partial V / \partial t$, per day)	$-[S N'(d_1) \sigma / (2\sqrt{T})] / 252 \pm r K e^{-rT} N(\pm d_2)$

where $N'(\cdot)$ is the standard normal PDF.

Implementation note: All BS functions are vectorised over NumPy arrays and broadcast correctly for scalar inputs, 1-D arrays, and mixed shapes. A `scalar_input` flag (True only when all inputs are 0-dimensional) determines whether to return a Python `float` or a NumPy array.

5. Volatility Surface Construction

5.1 ATM volatility — rolling realised vol proxy

Historical implied volatility data for equities is not available through `yfinance`. The model substitutes rolling annualised realised volatility as an ATM implied volatility proxy. This is a standard industry approximation; the implied-realised spread is relatively stable for liquid single-name equities.

Three rolling windows are computed (annualised by $\sqrt{252}$):

Label	Window (days)	Tenor range applied
short	21	$T < 3$ months ($T < 63/252$)
medium	63	$3 \text{ months} \leq T < 6 \text{ months}$
long	126	$T \geq 6 \text{ months}$

Formula:

$$\hat{\sigma}_{ATM}(t, W) = \sqrt{252} \cdot \text{RollingStd}\left(\ln \frac{S_t}{S_{t-1}}, W\right)$$

Warm-up period: The long window (126 days) requires 126 days of returns before the first valid estimate. The system enforces a `vol_warm_up = 126` day offset on all components that use rolling vols:

- Historical VAR (`historical_var.py`): test dates begin at index `lookback + vol_warm_up + 1 = 379` in the price series.
- Stress VAR (`stress_var.py`): the first 125 log-return rows (pre-warm-up) are trimmed before the vectorised stress scan.
- Rolling backtest (`backtest.py`): same `first_test = lookback + vol_warm_up + 1` offset applied to the test window.

No-arbitrage bounds: All ATM vol estimates are clipped:

`σ_ATM = clip(σ̂, VOL_FL00R, VOL_CAP)`

with `VOL_FL00R = 1e-4` (prevents zero/negative vol in BS) and `VOL_CAP = 5.0` (500% — prevents numerical explosion at extreme deep-OTM strikes). Full Dupire butterfly no-arbitrage constraints are not enforced; the quadratic skew form is well-behaved for $|k| < 1$ (typical liquid strikes).

5.2 Skew model — quadratic in log-moneyness

A quadratic function of log-moneyness k generates the full strike surface from the ATM level:

$$\sigma(K, T) = \sigma_{ATM}(T) + \alpha \cdot k + \beta \cdot k^2$$

where:

$$k = \ln\left(\frac{K}{F}\right), \quad F = S e^{rT}$$

Default parameters:

Parameter	Default	Interpretation
α	-0.15	Negative skew — downside puts more expensive than equidistant calls
β	0.05	Smile curvature — OTM options slightly richer than ATM

The same no-arbitrage clip is applied to the skewed output:

`σ(K, T) = clip(σ_ATM + α·k + β·k², VOL_FL00R, VOL_CAP)`

5.3 Surface lookup

The public API is `surface_vol(rolling_vols, ticker, date, K, S, r, T,  $\alpha$ ,  $\beta$ )`, which composes the two steps: tenor-matched ATM lookup followed by skew application.

6. Scenario Construction

6.1 Log-returns

Daily log-returns are computed from the price series:

$$r_{i,t} = \ln\left(\frac{S_{i,t}}{S_{i,t-1}}\right)$$

The first row (NaN from the initial shift) is dropped. The resulting DataFrame has shape $(T-1) \times N$ where T is the number of price observations and N is the number of tickers.

6.2 Scenario set

The scenario set is the most recent lookback (default: 252) rows of the log-returns DataFrame. Each row represents a historical joint one-day shock across all N tickers:

```
scenario_set = log_returns.iloc[-lookback:] # shape (252, N)
```

6.3 Stressed price matrix

Scenarios are applied to today's spot prices using the log-return identity:

$$S_{i,\text{stressed}}^{(t)} = S_{i,\text{today}} \cdot e^{r_{i,t}}$$

The result is a $(252, N)$ DataFrame (`scenario_prices`) where each row is a full joint stressed market state. The index carries the original historical dates (used to look up historical rolling vols during full revaluation).

6.4 Age weights

For Method B, exponentially decaying weights are assigned to the 252 scenarios. Indexing scenarios from oldest ($t=1$) to most recent ($t=N=252$):

$$w_t = \frac{(1 - \lambda) \lambda^{N-t}}{1 - \lambda^N}$$

The normalisation ensures $\sum_{t=1}^N w_t = 1$ exactly. Default $\lambda = 0.97$.

The weight on the most recent scenario relative to the oldest is:

$$\frac{w_N}{w_1} = \lambda^{-(N-1)} = 0.97^{-251} \approx 2,014$$

i.e., the most recent observation is ~2,000× more influential than the oldest.

7. Portfolio Revaluation

7.1 Full Black-Scholes revaluation

For each of the 252 scenarios, every option in the portfolio is repriced using the Black-Scholes formula under the stressed spot and the historical vol level on the scenario date:

$$V_{\text{stressed}}^{(t)} = \text{BS}\left(S_{\text{stressed}}^{(t)}, K, T_{\text{remaining}}, r, \sigma(K, T_{\text{remaining}}, \text{date}_t)\right)$$

The historical rolling vol from date_t is used for the ATM proxy. This captures actual historical vol dynamics (vol moves as the underlying moves), which is a key feature of the model — it goes beyond a simple spot-shock-only HS approach.

Trade P&L per scenario:

$$\Delta V^{(t)} = \left(V_{\text{stressed}}^{(t)} - V_{\text{today}}\right) \times \text{notional} \times \text{position_sign}$$

Portfolio P&L per scenario:

$$\text{PnL}^{(t)} = \sum_{\text{trades}} \Delta V^{(t)}$$

The result is a 1-D array of shape (252,). Losses are negative.

7.2 Greeks-based approximation (delta-gamma)

For the factor VAR decomposition (Method D), a faster delta-gamma approximation is used instead of full revaluation. For a spot change ΔS :

$$\Delta V \approx \left(\Delta \cdot \Delta S + \frac{1}{2}\Gamma \cdot (\Delta S)^2\right) \times \text{notional} \times \text{position_sign}$$

Greeks (Δ , Γ) are computed at today's market using the BS formulas in Section 4.2. Vega contribution is not modelled in the current implementation; this is noted as a known limitation in Section 17.

8. Risk Measures — VAR and ES

8.1 Value-at-Risk

Definition: $\text{VAR}(\alpha)$ is the loss not exceeded with probability α .

With N equal-probability scenarios, sorted in ascending order (most negative = largest loss first), the implementation uses:

$$\text{VAR}(\alpha) = -P_{\lfloor N(1-\alpha) \rfloor}$$

where P is the sorted P&L vector (1-indexed from the worst scenario).

Loss convention: VAR is reported as a positive number equal to the dollar loss at the quantile boundary.

For $N = 252$:

Confidence α	Tail count $\lfloor N(1-\alpha) \rfloor$	Scenario used
95%	12	12th worst
99%	2	2nd worst

8.2 Expected Shortfall

$\text{ES}(\alpha)$ is the mean loss over all scenarios in the tail:

$$\text{ES}(\alpha) = -\frac{1}{\lfloor N(1-\alpha) \rfloor} \sum_{t=1}^{\lfloor N(1-\alpha) \rfloor} P_t$$

$\text{ES} \geq \text{VAR}$ at all confidence levels. ES is a coherent risk measure (sub-additive); VAR is not.

8.3 Weighted VAR and ES

For age-weighted scenarios (Method B), VAR and ES use the weighted empirical CDF.

Sorting scenarios by P&L ascending and computing cumulative weights:

$$\text{VAR}_w(\alpha) = -P_{t^*}, \quad t^* = \min \left\{ t : \sum_{s=1}^t w_s \geq 1 - \alpha \right\}$$

$$\text{ES}_w(\alpha) = -\frac{\sum_{t:w_t \leq (1-\alpha) \text{ cumul.}} w_t \cdot P_t}{\sum_{t:w_t \leq (1-\alpha) \text{ cumul.}} w_t}$$

8.4 10-Day horizon scaling

The Basel square-root-of-time rule is applied to scale 1-day figures to a 10-day holding period:

$$\begin{aligned}\text{VAR}_{10d}(\alpha) &= \text{VAR}_{1d}(\alpha) \times \sqrt{10} \\ \text{ES}_{10d}(\alpha) &= \text{ES}_{1d}(\alpha) \times \sqrt{10}\end{aligned}$$

Limitation: This scaling assumes i.i.d. returns (no serial correlation, constant vol). In practice, it tends to underestimate risk during high-persistence vol regimes.

9. Method A — Plain Historical Simulation

Source: `src/var/historical_var.py` — `compute_historical_var(method="plain")`

Algorithm

1. Compute rolling vols and log-returns from the full price history.
2. Price the portfolio at `as_of` using today's market.
3. Slice the last 252 log-return rows to form the scenario set.
4. Apply each scenario to today's spots → 252 stressed price matrices.
5. Full BS reprice of all trades under each stressed state.
6. Compute portfolio P&L vector (252 values).
7. Apply `compute_var_es` to extract VAR and ES at each confidence level.
8. Scale to 10-day via $\sqrt{10}$.

Assumptions

- All 252 historical days are equally likely.
 - Options value changes are fully captured by BS repricing with historical vol dynamics.
 - No distributional assumption on returns (purely empirical).
-

10. Method B — Age-Weighted Historical Simulation

Source: `src/var/historical_var.py` — `compute_historical_var(method="age_weighted")`

Algorithm

Steps 1–6 are identical to Method A. Steps 7–8 differ:

7. Build age weights $w_t \propto \lambda^{(N-t)}$ for $t = 1 \dots N$, normalised to sum to 1.
8. Compute weighted VAR and ES using the weighted empirical CDF.
9. Scale to 10-day.

Motivation

The equal-weight assumption in Method A implies that a market shock from 252 days ago is as relevant as yesterday's move. Age-weighting with $\lambda < 1$ attenuates older observations, making the model more responsive to recent volatility regime changes. This is particularly valuable in periods following large market dislocations.

Sensitivity to λ

λ	Effective half-life (days)	Weight ratio (newest/oldest)
0.94	~11	5.9×
0.97	~23	45×
0.99	~69	~12×

Default $\lambda = 0.97$ gives a half-life of approximately 23 business days.

11. Method D — Factor-Based VAR

Source: `src/var/factor_var.py` — `compute_factor_var`

11.1 Decomposition

Equity log-returns are decomposed into a systematic (factor-driven) component and an idiosyncratic residual via OLS regression:

$$r_{i,t} = \alpha_i + \sum_{k=1}^K \beta_{i,k} \cdot F_{k,t} + \varepsilon_{i,t}$$

where: - $F_{k,t}$ are market index log-returns (e.g., S&P 500) on day t - $\beta_{i,k}$ are factor loadings (OLS estimates) - $\varepsilon_{i,t}$ are idiosyncratic residuals (zero-mean by construction)

The OLS is solved using `numpy.linalg.lstsq` on the full historical return history.

11.2 Factor betas

The `fit_factor_model` function returns a DataFrame with index = equity tickers and columns = [alpha, factor names, r_squared].

Example output (SPX as single factor):

Ticker	alpha	SPX	r_squared
AAPL	0.0002	0.82	0.52
MSFT	0.0001	0.91	0.61
...	...	...	...

11.3 Systematic P&L

For each factor scenario (last 252 factor log-returns), the implied equity spot change for ticker i is:

$$\Delta S_i^{\text{sys}} = S_i \left(\exp \left(\sum_k \beta_{i,k} F_{k,t} \right) - 1 \right)$$

The systematic portfolio P&L is computed via delta-gamma approximation (Section 7.2):

$$\text{PnL}_{\text{sys}}^{(t)} = \sum_{\text{trades}} \left(\Delta_j \cdot \Delta S_{i(j)}^{\text{sys}} + \frac{1}{2} \Gamma_j \cdot (\Delta S_{i(j)}^{\text{sys}})^2 \right) \times \text{notional}_j \times \text{sign}_j$$

11.4 Idiosyncratic P&L

OLS residuals are computed as:

$$\varepsilon_{i,t} = r_{i,t} - \left(\alpha_i + \sum_k \beta_{i,k} F_{k,t} \right)$$

The residual scenarios are date-aligned to the factor scenarios (using index intersection). The idiosyncratic spot change is:

$$\Delta S_i^{\text{idio}} = S_i (e^{\varepsilon_{i,t}} - 1)$$

Idiosyncratic P&L uses the same delta-gamma approximation applied to ΔS_i^{idio} .

11.5 Combined P&L

The combined P&L for each scenario is the sum of both components:

$$\text{PnL}_{\text{total}}^{(t)} = \text{PnL}_{\text{sys}}^{(t)} + \text{PnL}_{\text{idio}}^{(t)}$$

This is computed scenario-by-scenario, preserving the joint distribution between systematic and idiosyncratic shocks. This is superior to simple Gaussian aggregation (which would assume independence between components and use $\text{VAR}_{\text{total}}^2 \approx \text{VAR}_{\text{sys}}^2 + \text{VAR}_{\text{idio}}^2$).

11.6 Diversification benefit

Because systematic and idiosyncratic components can partially offset each other scenario-by-scenario, the combined VAR is typically less than the naive sum:

$$\text{VAR}_{\text{total}} \leq \text{VAR}_{\text{sys}} + \text{VAR}_{\text{idio}}$$

The difference is the diversification benefit from the factor structure.

12. Stressed VAR — Basel 2.5

Source: `src/var/stress_var.py` — `compute_stress_var`

12.1 Concept

Basel 2.5 requires that the internal model capital charge include a Stressed VAR — the VAR that would have been produced by the current portfolio during the worst 252-day period in the full historical price history.

12.2 Algorithm

Vectorised scan ($O(N)$ pricing complexity):

1. Trim log-returns to post-warm-up dates: `log_returns = log_returns.iloc[vol_warm_up - 1:]` (removes first 125 rows where 126-day rolling vol is NaN).
2. Build the stressed price matrix for all remaining N scenarios at once: `all_scenario_prices = build_stressed_price_matrix(today_prices, log_returns)`
3. Reprice the full portfolio under all N scenarios in one pass: `all_pnl = full_reprice_pnl(...)` — shape $(N,)$
4. Construct the $(n_windows \times 252)$ windowed P&L matrix via NumPy stride trick:

```
n_windows = N - 252 + 1
row_idx = np.arange(252)[np.newaxis, :] + np.arange(n_windows)[:, np.newaxis]
windowed_pnl = all_pnl[row_idx]           # shape (n_windows, 252)
```
5. Compute VAR for each rolling window (sort ascending, take n_tail -th value):

```
sorted_windows = np.sort(windowed_pnl, axis=1)
window_vars = -sorted_windows[:, n_tail - 1]  # VAR per window (positive)
```
6. Select the window with the highest VAR:

```
best_start_idx = int(np.argmax(window_vars))
```
7. Compute VAR, ES and 10-day scaled figures over the selected 252-day window.

Computational complexity: Step 3 performs N BS pricings (vectorised over scenarios). The naive loop approach would require $n_windows \times 252$ BS pricings. For a 1 300-day history, this is 1 175 vs ~277 000 pricings — approximately a 235× reduction.

12.3 Output

```
{
  "method":      "stress_var",
  "stress_start": pd.Timestamp,  # start of worst window
  "stress_end":   pd.Timestamp,  # end of worst window
  "n_scenarios":  int,           # 252
  "as_of":        pd.Timestamp,
  "pnl_vector":   np.ndarray,    # P&L in the stress window
  "var_95":       float,
```

```

    "es_95": float,
    "var_99": float,
    "es_99": float,
    "var_95_10d": float,
    "es_95_10d": float,
    "var_99_10d": float,
    "es_99_10d": float,
}

```

13. Rolling Backtest

Source: `src/backtest/backtest.py` — `rolling_backtest`, `backtest_report`

13.1 Procedure

For each business day `as_of` in the test window:

1. Trade aging: Filter the portfolio to active trades (expiry strictly after `as_of`). Expired trades are excluded from both VAR and realised P&L computation.
2. VAR computation: Build the 252-day scenario set ending at `as_of`; full-reprice the active portfolio; compute VAR at each confidence level.
3. Realised P&L: Price the same active portfolio at `as_of + 1`; subtract today's value:

$$\text{PnL}_{\text{realised}} = \sum_j V_j(t+1) - \sum_j V_j(t)$$

4. Exception flag: An exception occurs when the realised loss exceeds VAR:

$$\mathbb{1}_{\text{exception}} = \mathbb{1}[\text{PnL}_{\text{realised}} < -\text{VAR}(\alpha)]$$

5. Record `{date, var_α, es_α, realised_pnl, exception_α, n_active_trades}` for each test date.

13.2 Warm-up offset

The test window starts at index `lookback + vol_warm_up + 1 = 379` in the price series (`252 + 126 + 1`). This ensures: - Enough history for the 252-day scenario set. - All scenario dates within that set fall after the vol warm-up period (no NaN vols).

13.3 Empty book handling

If all trades have expired on `as_of`, the backtest records zero VAR, zero ES, and zero realised P&L for that date (no exception).

14. Statistical Tests

14.1 Kupiec Proportion of Failures (POF) test

Source: `src/utils/stats.py` — `kupiec_test`

Null hypothesis: The true exception rate equals the expected rate $p = 1 - \alpha$.

The likelihood ratio statistic follows a $\chi^2(1)$ distribution under H_0 :

$$LR_{\text{POF}} = 2 \left[x \ln \left(\frac{\hat{p}}{p} \right) + (n - x) \ln \left(\frac{1 - \hat{p}}{1 - p} \right) \right] \xrightarrow{d} \chi^2(1)$$

where x = number of exceptions, n = observations, $\hat{p} = x/n$.

Edge case $x = 0$: The MLE $\hat{p} = 0$ makes the general formula undefined. The exact limit is:

$$LR = 2n \ln \left(\frac{1}{1 - p} \right)$$

This correctly rejects an over-conservative model under the $\chi^2(1)$ distribution. A `UserWarning` is issued when $x = 0$ to flag potentially over-conservative models.

Rejection criterion: Reject H_0 if p-value < 0.05 (5% significance level).

14.2 Christoffersen Conditional Coverage / Independence Test

Source: `src/utils/stats.py` — `christoffersen_test`

Null hypothesis: The exception process is i.i.d. (no temporal clustering).

Transition counts are extracted from the binary exception series:

	Next day: 0	Next day: 1
Today: 0	n_{00}	n_{01}
Today: 1	n_{10}	n_{11}

Transition probabilities:

$$\hat{p}_{01} = \frac{n_{01}}{n_{00} + n_{01}}, \quad \hat{p}_{11} = \frac{n_{11}}{n_{10} + n_{11}}, \quad \hat{p} = \frac{n_{01} + n_{11}}{n - 1}$$

The independence LR statistic:

$$LR_{\text{ind}} = 2 [\ell(\hat{p}_{01}, n_{00}, n_{01}) + \ell(\hat{p}_{11}, n_{10}, n_{11}) - \ell(\hat{p}, n_{00} + n_{10}, n_{01} + n_{11})] \xrightarrow{d} \chi^2(1)$$

where $\ell(p, n_0, n_1) = n_0 \ln(1 - p) + n_1 \ln(p)$.

Numerical safety: LR is floored at 0 to prevent negative values from numerical precision issues when log-likelihoods are nearly equal.

14.3 Basel Traffic Light Classification

Source: `src/utils/stats.py` — `traffic_light`

Applied to the number of exceptions over a 250-day backtest window:

Exceptions	Zone	Regulatory action
0 – 4	Green	No capital add-on
5 – 9	Amber	Progressive capital add-on
10+	Red	Presumptive model rejection

15. Trade Lifecycle Management

Source: `src/backtest/backtest.py` — `_active_trades`

On each backtest date `as_of`, options are filtered by:

```
active = trades[pd.to_datetime(trades["expiry"]) > as_of]
```

A trade with `expiry = as_of` is not active on that date (strict inequality). This correctly models options that expired at the close of the previous business day.

Cash settlement proceeds from expired options are not tracked — this is noted as a model scope limitation (out-of-scope for the VAR engine, which focuses on mark-to-market changes of live positions).

16. Implementation Engineering Notes

16.1 Warm-up period management

Three components depend on the 126-day rolling vol warm-up:

Component	First valid index	How enforced
<code>get_atm_vol</code>	<code>prices.index[126]</code>	Raises <code>ValueError</code> on NaN
<code>rolling_backtest</code>	<code>prices.index[379]</code>	<code>first_test = lookback + vol_warm_up + 1</code>
<code>find_stress_window</code>	<code>log_returns.index[125]</code>	<code>log_returns = log_returns.iloc[vol_warm_up - 1:]</code>

16.2 Vectorisation strategy

Full portfolio revaluation (`full_reprice_pnl`) loops over trades and scenarios. The outer loop is over trades (`N_trades = 50` typical), inner over scenarios (252). For Stress VAR, all `N` historical scenarios (up to ~1 175 post-warm-up) are priced in one pass to avoid $O(n_windows \times 252)$ redundant BS calls.

For the factor VAR approximation, delta-gamma P&L is vectorised using NumPy array operations across all scenarios simultaneously (no Python loop over scenarios).

16.3 Numerical conventions

Convention	Value
Trading days per year	252
Time to expiry	Calendar days / 365.25 (not business days)
P&L sign	Losses negative in raw P&L; VAR and ES reported positive
$\sqrt{10}$ scaling constant	Pre-computed as <code>SQRT_10 = np.sqrt(10)</code>

16.4 Configuration management

All key parameters are passed explicitly at call-site. There is no global mutable configuration. Default values are expressed as Python function defaults:

Parameter	Default	Location
<code>confidence_levels</code>	[0.95, 0.99]	All VAR functions
<code>lookback</code>	252	<code>compute_historical_var</code> , <code>rolling_backtest</code>
<code>lambda_</code>	0.97	<code>age_weighted_var</code>
<code>r</code>	0.05	All pricing functions
<code>alpha (skew)</code>	-0.15	<code>generate_portfolio</code> , <code>price_portfolio</code>
<code>beta (smile)</code>	0.05	<code>generate_portfolio</code> , <code>price_portfolio</code>
<code>window</code>	252	<code>compute_stress_var</code>

17. Model Limitations and Known Constraints

#	Limitation	Severity	Mitigation / Future work
1	Realised vol as IV proxy — ignores jumps in implied vol (e.g., VIX spikes on event days)	Medium	GARCH-filtered HS (Method C) partially addresses this; alternatively, calibrate α , β to current options chain
2	No vega P&L in delta-gamma approximation — Method D underestimates P&L for large vol moves	Medium	Extend to delta-gamma-vega; or use full revaluation for all methods
3	$\sqrt{10}$ scaling assumes i.i.d. returns — overestimates horizon risk in low-vol regimes, underestimates in trending regimes	Low-Medium	Multi-day overlapping scenarios (non-overlapping 10-day) or filtered HS

#	Limitation	Severity	Mitigation / Future work
4	European options only — American options require different pricing	Low	Extend to binomial tree or Barone-Adesi-Whaley for American options
5	Factor model treats index options as missing — trades on index underlyings are skipped in factor P&L	Low	Assign beta = 1 to own index; extend compute_factor_pnl
6	Cash settlement not tracked — expired options leave a gap in realised P&L continuity	Low	Out of scope for VAR engine; relevant for P&L attribution
7	No cross-gamma terms — delta-gamma approximation ignores cross-spot Greeks between different underlyings	Low	Acceptable for single-name options; relevant for multi-asset structures
8	Dupire no-arbitrage not enforced — quadratic skew can violate butterfly constraints far from ATM	Low	Enforce $\partial^2 C / \partial K^2 \geq 0$ constraint; or cap
9	OLS factor model — assumes linear, time-invariant betas; does not capture regime shifts	Medium	Rolling OLS, Kalman filter betas, or Bayesian updating
10	Single risk-free rate — flat yield curve across all maturities	Low	Use tenor-matched rates from a yield curve

18. Parameter Reference

Global constants (**vol_surface.py**)

```

VOL_WINDOWS = {"short": 21, "medium": 63, "long": 126} # days
VOL_FLOOR   = 1e-4   # minimum vol (no-arbitrage lower bound)
VOL_CAP     = 5.0     # maximum vol (500%, numerical safety cap)

```

Tenor cutoffs (`vol_surface.py`)

```
_SHORT_CUTOFF = 63 / 252 # 3 months
_MEDIUM_CUTOFF = 126 / 252 # 6 months
```

Warm-up derived constants

```
vol_warm_up = max(VOL_WINDOWS.values()) # 126
first_test = lookback + vol_warm_up + 1 # 379 (for lookback=252)
```

Default skew parameters

```
DEFAULT_ALPHA = -0.15 # vol_surface.py
DEFAULT_BETA = 0.05
```

Age-weighting

```
lambda_ = 0.97 # default decay factor
N_scenarios = 252
```

Backtesting

```
significance_level = 0.05 # Kupiec / Christoffersen rejection threshold
# Traffic light thresholds (Basel, 250-day window):
GREEN_MAX = 4
AMBER_MAX = 9
# RED: >= 10
```

19. Module Reference

Module	Key public functions / classes
<code>src/utils/black_scholes.py</code>	<code>bs_price</code> , <code>bs_delta</code> , <code>bs_gamma</code> , <code>bs_vega</code> , <code>bs_theta</code> , <code>bs_greeks</code>
<code>src/utils/stats.py</code>	<code>compute_var_es</code> , <code>compute_weighted_var_es</code> , <code>scale_to_10d</code> , <code>weighted_quantile</code> , <code>kupiec_test</code> , <code>christoffersen_test</code> , <code>traffic_light</code>
<code>src/data/loader.py</code>	<code>load_prices</code> , <code>_download_single</code> (internal)
<code>src/data/vol_surface.py</code>	<code>compute_rolling_vol</code> , <code>get_atm_vol</code> , <code>skew_vol</code> , <code>surface_vol</code>
<code>src/data/portfolio.py</code>	<code>validate_trades</code> , <code>load_portfolio</code> , <code>price_trade</code> , <code>price_portfolio</code> , <code>portfolio_greeks</code> , <code>generate_portfolio</code>
<code>src/var/scenarios.py</code>	<code>compute_log_returns</code> , <code>build_scenario_set</code> , <code>build_age_weights</code> , <code>build_stressed_price_matrix</code>
<code>src/var/revaluation.py</code>	<code>reprice_pnl</code> , <code>greeks_approx_pnl</code>
<code>src/var/historical.py</code>	<code>plain_var</code> , <code>age_weighted_var</code> , <code>compute_historical_var</code>

Module	Key public functions / classes
src/var/factor_var.py	factor_model, compute_factor_pnl, compute_residuals, compute_idio_pnl, compute_factor_var
src/var/stress_var.py	find_stress_window, compute_stress_var
src/backtest/backtest.py	estimate_trades, rolling_backtest, count_exceptions, backtest_report

End of Technical Specification